POO avancée et Design Patterns

Arbre binaire de recherche et std : :map

Saad El-Assali — Filière ILISI
Professeur : Adil El Bouziri | Année universitaire : 2026–2027

Table des matières

1 Problématique **2**
1.1 Objectifs 2

2 Principe de l'arbre binaire de recherche **2**

3 Implémentation **2**
3.1 Programme de l'arbre binaire 2
3.2 Programme avec std : :map 4

4 Comparaison **5**
4.1 Calcul des complexités 5
4.2 Mesures numériques et visualisation 5

5 Conclusion **6**

1 Problématique

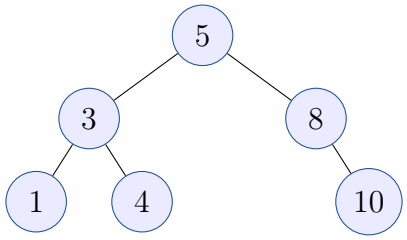
Ce travail consiste à implémenter un arbre binaire de recherche en C++, puis à comparer son fonctionnement avec la structure standard `std::map`.

1.1 Objectifs

— Comprendre le fonctionnement d'un arbre binaire de recherche.
— Implémenter l'insertion et la recherche avec des nœuds et des pointeurs.
— Utiliser `std::map` comme structure standard équilibrée.
— Comparer les opérations et leurs complexités théoriques.

2 Principe de l'arbre binaire de recherche

Pour chaque nœud, les valeurs plus petites ou égales sont placées dans le sous-arbre gauche et les valeurs plus grandes dans le sous-arbre droit.



Une lecture infixe parcourt le sous-arbre gauche, le nœud, puis le sous-arbre droit. Elle affiche donc les valeurs dans l'ordre croissant.

3 Implémentation

Chaque élément est représenté par un objet `Noeud` contenant une valeur et deux pointeurs. La fonction `inserer` est récursive : elle descend dans l'arbre jusqu'à trouver une position vide. La fonction `rechercher` compare la valeur demandée avec le nœud courant et choisit le côté approprié.

3.1 Programme de l'arbre binaire

```
#include <cstdlib>
#include <iostream>

class Noeud {
public:
  int valeur;
  Noeud *fd;
  Noeud *fg;

  Noeud(int v) {
    valeur = v;
    fd = nullptr;
    fg = nullptr;
```

```
14    }
15  };
16
17  class ArbreBin {
18  public:
19    Noeud *racine = nullptr;
20
21    Noeud *inserer(Noeud *courant, int val) {
22      if (!courant) {
23        return new Noeud(val);
24      }
25      if (val <= courant->valeur) {
26        courant->fg = inserer(courant->fg, val);
27      } else if (val > courant->valeur) {
28        courant->fd = inserer(courant->fd, val);
29      }
30      return courant;
31    }
32
33    void afficher(Noeud *courant) {
34      if (courant) {
35        afficher(courant->fg);
36        std::cout << courant->valeur << " ";
37        afficher(courant->fd);
38      }
39    }
40
41    void detruire(Noeud *courant) {
42      if (courant) {
43        detruire(courant->fg);
44        detruire(courant->fd);
45        delete courant;
46      }
47    }
48
49    ~ArbreBin() { detruire(racine); }
50
51    ArbreBin() = default;
52    ArbreBin(const ArbreBin &) = delete;
53    ArbreBin &operator=(const ArbreBin &) = delete;
54  };
55
56  int main() {
57    ArbreBin arbre;
58    int choix;
59    int val;
60
61    while (true) {
62      std::cout << "\n1. Inserer\n"
63                << "2. Afficher\n"
64                << "3. Inserer 1M valeurs aleatoires\n"
65                << "0. Quitter\n"
66                << "Choix: ";
67      std::cin >> choix;
68
69      switch (choix) {
70      case 1:
71        std::cout << "Valeur: ";
```

```
72        std::cin >> val;
73        arbre.racine = arbre.inserer(arbre.racine, val);
74        break;
75      case 2:
76        std::cout << "\n";
77        arbre.afficher(arbre.racine);
78        std::cout << "\n";
79        break;
80      case 3:
81        for (int i = 0; i < 1000000; i++) {
82          arbre.racine = arbre.inserer(arbre.racine, std::rand() %
                1000000);
83        }
84        std::cout << "1000000 valeurs inserees\n";
85        break;
86      case 0:
87        return 0;
88      default:
89        std::cout << "Choix invalide\n";
90        break;
91      }
92    }
93  }
```

3.2 Programme avec std : :map

```
1   #include <cstdlib>
2   #include <iostream>
3   #include <map>
4
5   int main() {
6     std::map<int, int> arbre;
7     int choix, val;
8
9     while (true) {
10      std::cout << "\n1. Inserer\n2. Afficher\n3. Inserer 1M valeurs
            aleatoires\n0. Quitter\nChoix: ";
11      if (!(std::cin >> choix)) return 0;
12
13      switch (choix) {
14      case 1:
15        std::cout << "Valeur: ";
16        if (!(std::cin >> val)) return 0;
17        arbre[val]++;
18        break;
19      case 2:
20        for (const auto &element : arbre)
21          for (int i = 0; i < element.second; i++)
22            std::cout << element.first << ' ';
23        std::cout << '\n';
24        break;
25      case 3:
26        for (int i = 0; i < 1000000; i++)
27          arbre[std::rand() % 1000000]++;
28        std::cout << "1000000 valeurs inserees\n";
29        break;
```

```
30      case 0:
31        return 0;
32      default:
33        std::cout << "Choix invalide\n";
34      }
35    }
36  }
```

Dans la version utilisant `std::map`, la clé est la valeur et l'entier associé représente le nombre d'occurrences. Cela permet de conserver les doublons comme dans l'arbre personnel.

4 Comparaison

Critère	Arbre personnel	`std::map`
Insertion / recherche	$O(h)$	$O(\log n)$
Pire cas	$O(n)$	$O(\log n)$
Doublons	Un nœud par valeur	Compteur par clé

Ici, h désigne la hauteur de l'arbre. Un arbre non équilibré peut devenir une chaîne, tandis que `std::map` garantit une complexité logarithmique pour l'insertion et la recherche.

4.1 Calcul des complexités

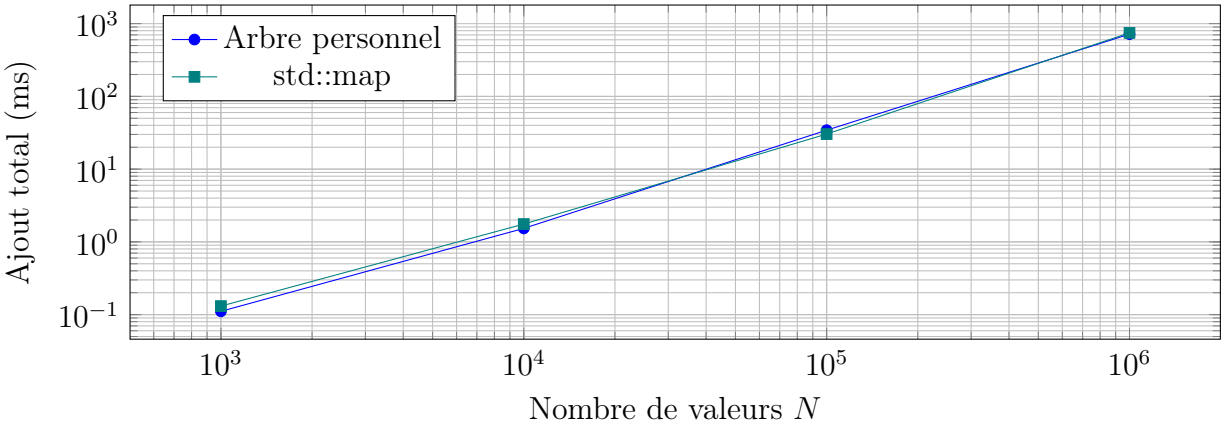
Une insertion ou une recherche visite au plus un nœud par niveau : $T(n) = O(h)$. Pour un arbre équilibré, $h = O(\log n)$, donc $T(n) = O(\log n)$. Dans le pire cas, l'arbre personnel est une chaîne de hauteur n, donc $T(n) = O(n)$. Le parcours infixe visite chaque nœud une fois : $T(n) = O(n)$.

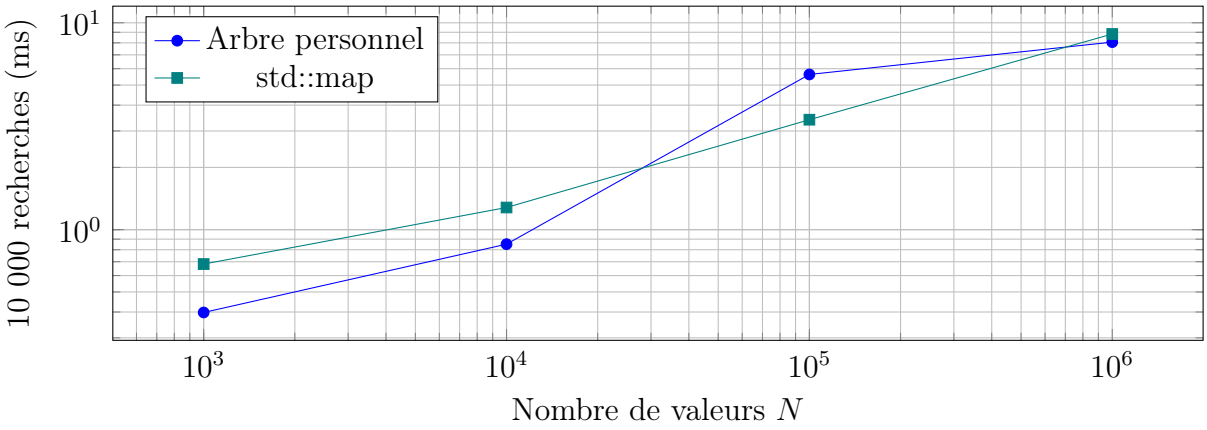
4.2 Mesures numériques et visualisation

Les deux structures reçoivent les mêmes valeurs aléatoires et les mêmes 10 000 requêtes. La génération des données est hors chronométrage. Mesures d'une exécution sur Intel i7-7600U avec `g++ -O2` ; elles varient selon la charge de la machine. La vérification compare le nombre total de recherches réussies, sans comparer chaque réponse individuellement.

N	Ajout arbre	Ajout map	Recherche arbre	Recherche map
1 000	0.111	0.131	0.398	0.682
10 000	1.533	1.758	0.851	1.280
100 000	34.315	30.348	5.628	3.401
1 000 000	717.601	746.578	8.066	8.827

Tous les temps du tableau sont en millisecondes (ms) ; les ajouts portent sur N valeurs et les recherches sur 10 000 requêtes. Comme 1 s = 1000 ms et 1 ms = 10^6 ns, 717.601 ms correspondent à 0.717601 s. Pour $N = 1\,000\,000$, le temps moyen des recherches dans l'arbre est $8.066 \times 10^6 / 10\,000 = 806.6$ ns par requête ; c'est une moyenne, pas la durée de chaque requête.





5 Conclusion

L'arbre binaire de recherche permet de comprendre directement les nœuds, les pointeurs et la récursion. Il est efficace lorsque sa hauteur reste faible, mais il peut devenir une chaîne dans le pire cas. `std::map` garantit une complexité logarithmique pour la recherche et l'insertion.